

FRED TALKS

7th May 2026

CONTENTS

Highlights from my conversation about agentic engineering on
Lenny's Podcast

SIMON WILLISON'S WEBLOG: ENTRIES · 3349 WORDS

A PM's Guide to AI Agent Architecture: Why Capability Doesn't
Equal Adoption

UMANG · 1871 WORDS

Highlights from my conversation about agentic engineering on Lenny's Podcast

SIMON WILLISON'S WEBLOG: ENTRIES · 02 APR 2026 · [SOURCE](#)

Highlights from my conversation about agentic engineering on Lenny's Podcast

I was a guest on Lenny Rachitsky's podcast, in a new episode titled [An AI state of the union: We've passed the inflection point, dark factories are coming, and automation timelines](#). It's available on [YouTube](#), [Spotify](#), and [Apple Podcasts](#). Here are my highlights from our conversation, with relevant links.

[An AI state of the union: We've passed the inflection point & dark factories are coming](#)[Lenny's Podcast](#)



Lenny's Podcast 571K subscribers

[Watch on](#)

- [The November inflection point](#)
- [Software engineers as bellwethers for other information workers](#)
- [Writing code on my phone](#)
- [Responsible vibe coding](#)
- [Dark Factories and StrongDM](#)
- [The bottleneck has moved to testing](#)
- [This stuff is exhausting](#)
- [Interruptions cost a lot less now](#)

- My ability to estimate software is broken
- It's tough for people in the middle
- It's harder to evaluate software
- The misconception that AI tools are easy
- Coding agents are useful for security research now
- OpenClaw
- Journalists are good at dealing with unreliable sources
- The pelican benchmark
- And finally, some good news about parrots
- YouTube chapters

The November inflection point

4:19—The end result of these two labs throwing everything they had at making their models better at code is that in November we had what I call the inflection point where GPT 5.1 and Claude Opus 4.5 came along.

They were both incrementally better than the previous models, but in a way that crossed a threshold where previously the code would mostly work, but you had to pay very close attention to it. And suddenly we went from that to... almost all of the time it does what you told it to do, which makes all of the difference in the world.

Now you can spin up a coding agent and say, build me a Mac application that does this thing, and you'll get something back which won't just be a buggy pile of rubbish that doesn't do anything.

Software engineers as bellwethers for other information workers

5:49—I can churn out 10,000 lines of code in a day. And most of it works. Is that good? Like, how do we get from most of it works to all of it works? There are so many new questions that we’re facing, which I think makes us a bellwether for other information workers.

Code is easier than almost every other problem that you pose these agents because code is obviously right or wrong—either it works or it doesn’t work. There might be a few subtle hidden bugs, but generally you can tell if the thing actually works.

If it writes you an essay, if it prepares a lawsuit for you, it’s so much harder to derive if it’s actually done a good job, and to figure out if it got things right or wrong. But it’s happening to us as software engineers. It came for us first.

And we’re figuring out, OK, what do our careers look like? How do we work as teams when part of what we did that used to take most of the time doesn’t take most of the time anymore? What does that look like? And it’s going to be very interesting seeing how this rolls out to other information work in the future.

Lawyers are falling for this really badly. The [AI hallucination cases database](#) is up to 1,228 cases now!

Plus this bit from the cold open at [the start](#):

It used to be you’d ask ChatGPT for some code, and it would spit out some code, and you’d have to run it and test it. The coding agents take that step for you now. And an open question for me is how many other knowledge work fields are actually prone to these agent loops?

Writing code on my phone

8:19—I write so much of my code on my phone. It’s wild. I can get good work done walking the dog along the beach, which is delightful.

I mainly use the Claude iPhone app for this, both with a regular Claude chat session (which [can execute code now](#)) or using it to control [Claude Code for web](#).

Responsible vibe coding

9:55 If you're vibe coding something for yourself, where the only person who gets hurt if it has bugs is you, go wild. That's completely fine. The moment you ship your vibe coding code for other people to use, where your bugs might actually harm somebody else, that's when you need to take a step back.

See also [When is it OK to vibe code?](#)

Dark Factories and StrongDM

12:49 The reason it's called the dark factory is there's this idea in factory automation that if your factory is so automated that you don't need any people there, you can turn the lights off. Like the machines can operate in complete darkness if you don't need people on the factory floor. What does that look like for software? [...]

So there's this policy that nobody writes any code: you cannot type code into a computer. And honestly, six months ago, I thought that was crazy. And today, probably 95% of the code that I produce, I didn't type myself. That world is practical already because the latest models are good enough that you can tell them to rename that variable and refactor and add this line there... and they'll just do it—it's faster than you typing on the keyboard yourself.

The next rule though, is nobody *reads* the code. And this is the thing which StrongDM started doing last year.

I wrote a lot more about [StrongDM's dark factory explorations](#) back in February.

The bottleneck has moved to testing

21:27—It used to be, you'd come up with a spec and you hand it to your engineering team. And three weeks later, if you're lucky, they'd come back with an implementation. And now that maybe takes three hours, depending on how well the coding agents are established for that kind of thing. So now what, right? Now, where else are the bottlenecks?

Anyone who's done any product work knows that your initial ideas are always wrong. What matters is proving them, and testing them.

We can test things so much faster now because we can build workable prototypes so much quicker. So there's an interesting thing I've been doing in my own work where any feature that I want to design, I'll often prototype three different ways it could work because that takes very little time.

I've always loved prototyping things, and prototyping is even more valuable now.

22:40—A UI prototype is free now. ChatGPT and Claude will just build you a very convincing UI for anything that you describe. And that's how you should be working. I think anyone who's doing product design and isn't vibe coding little prototypes is missing out on the most powerful boost that we get in that step.

But then what do you do? Given your three options that you have instead of one option, how do you prove to yourself which one of those is the best? I don't have a confident answer to that. I expect this is where the good old fashioned usability testing comes in.

More on prototyping later on:

46:35—Throughout my entire career, my superpower has been prototyping. I've been very quick at knocking out working prototypes of things. I'm the person who can show up at a meeting and say, look, here's how it could work. And that was kind of my unique selling point. And that's gone. Anyone can do what I could do.

This stuff is exhausting

26:25—I'm finding that using coding agents well is taking every inch of my 25 years of experience as a software engineer, and it is mentally exhausting. I can fire up four agents in parallel and have them work on four different problems. And by like 11 AM, I am wiped out for the day. [...]

There's a personal skill we have to learn in finding our new limits—what's a responsible way for us not to burn out.

I've talked to a lot of people who are losing sleep because they're like, my coding agents could be doing work for me. I'm just going to stay up an extra half hour and set off a bunch of extra things... and then waking up at four in the morning. That's obviously unsustainable. [...]

There's an element of sort of gambling and addiction to how we're using some of these tools.

Interruptions cost a lot less now

45:16—People talk about how important it is not to interrupt your coders. Your coders need to have solid two to four hour blocks of uninterrupted work so they can spin up their mental model and churn out the code. That's changed completely. My programming work, I need two minutes every now and then to prompt my agent about what to do next. And then I can do the other stuff and I can go back. I'm much more interruptible than I used to be.

My ability to estimate software is broken

28:19—I've got 25 years of experience in how long it takes to build something. And that's all completely gone—it doesn't work anymore because I can look at a problem and say that this is going to take two weeks, so it's not worth it. And now it's like... maybe it's going to take 20 minutes because the reason it would have taken two weeks was all of the sort of cruffy coding things that the AI is now covering for us.

I constantly throw tasks at AI that I don't think it'll be able to do because every now and then it does it. And when it doesn't do it, you learn, right? But when it *does* do something, especially something that the previous models couldn't do, that's actually cutting edge AI research.

And a related anecdote:

36:56—A lot of my friends have been talking about how they have this backlog of side projects, right? For the last 10, 15 years, they've got projects they never quite finished. And some of them are like, well, I've done them all now. Last couple of months, I just went through and every evening I'm like, let's take that project and finish it. And they almost feel a sort of sense of loss at the end where they're like, well, okay, my backlog's gone. Now what am I going to build?

It's tough for people in the middle

29:29—So ThoughtWorks, the big IT consultancy, did an offsite about a month ago, and they got a whole bunch of engineering VPs in from different companies to talk about this stuff. And one of the interesting theories they came up with is they think this stuff is really good for experienced engineers, like it amplifies their skills. It's really good for new engineers because it solves so many of those onboarding problems. The problem is

the people in the middle. If you're mid-career, if you haven't made it to sort of super senior engineer yet, but you're not sort of new either, that's the group which is probably in the most trouble right now.

I mentioned [Cloudflare hiring 1,000 interns](#), and Shopify too.

Lenny asked for my advice for people stuck in that middle:

31:21—That's a big responsibility you're putting on me there! I think the way forward is to lean into this stuff and figure out how do I help this make me better?

A lot of people worry about skill atrophy: if the AI is doing it for you, you're not learning anything. I think if you're worried about that, you push back at it. You have to be mindful about how you're applying the technology and think, okay, I've been given this thing that can answer any question and *often* gets it right. How can I use this to amplify my own skills, to learn new things, to take on much more ambitious projects? [...]

33:05—Everything is changing so fast right now. The only universal skill is being able to roll with the changes. That's the thing that we all need.

The term that comes up most in these conversations about how you can be great with AI is *agency*. I think agents have no agency at all. I would argue that the one thing AI can never have is agency because it doesn't have human motivations.

So I'd say that's the thing is to invest in your own agency and invest in how to use this technology to get better at what you do and to do new things.

It's harder to evaluate software

The fact that it's so easy to create software with detailed documentation and robust tests means it's harder to figure out what's a credible project.

37:47 Sometimes I'll have an idea for a piece of software, Python library or whatever, and I can knock it out in like an hour and get to a point where it's got documentation and tests and all of those things, and it looks like the kind of software that previously I'd have spent several weeks on—and I can stick it up on GitHub

And yet... I don't believe in it. And the reason I don't believe in it is that I got to rush through all of those things... I think the quality is probably good, but I haven't spent enough time with it to feel confident in that quality. Most importantly, *I haven't used it yet*.

It turns out when I'm using somebody else's software, the thing I care most about is I want them to have used it for months.

I've got some very cool software that I built that I've *never used*. It was quicker to build it than to actually try and use it!

The misconception that AI tools are easy

41:31—Everyone's like, oh, it must be easy. It's just a chat bot. It's not easy. That's one of the great misconceptions in AI is that using these tools effectively is easy. It takes a lot of practice and it takes a lot of trying things that didn't work and trying things that did work.

Coding agents are useful for security research now

19:04—In the past sort of three to six months, they've started being credible as security researchers, which is sending shockwaves through the security research industry.

See Thomas Ptacek: [Vulnerability Research Is Cooked](#).

At the same time, open source projects are being bombarded with junk security reports:

20:05—There are these people who don't know what they're doing, who are asking ChatGPT to find a security hole and then reporting it to the maintainer. And the report looks good. ChatGPT can produce a very well formatted report of a vulnerability. It's a total waste of time. It's not actually verified as being a real problem.

A good example of the right way to do this is [Anthropic's collaboration with Firefox](#), where Anthropic's security team *verified* every security problem before passing them to Mozilla.

OpenClaw

Of course we had to talk about OpenClaw! Lenny had his running on a Mac Mini.

1:29:23—OpenClaw demonstrates that people want a personal digital assistant so much that they are willing to not just overlook the security side of things, but also getting the thing running is not easy. You’ve got to create API keys and tokens and install stuff. It’s not trivial to get set up and hundreds of thousands of people got it set up. [...]

The first line of code for OpenClaw was written on November the 25th. And then in the Super Bowl, there was an ad for AI.com, which was effectively a vaporware white labeled OpenClaw hosting provider. So we went from first line of code in November to Super Bowl ad in what? Three and a half months.

I continue to love Drew Breunig’s description of OpenClaw as a digital pet:

A friend of mine said that OpenClaw is basically a Tamagotchi. It’s a digital pet and you buy the Mac Mini as an aquarium.

Journalists are good at dealing with unreliable sources

In talking about my explorations of AI for data journalism through Datasette:

1:34:58—You would have thought that AI is a very bad fit for journalism where the whole idea is to find the truth. But the flip side is journalists deal with untrustworthy sources all the time. The art of journalism is you talk to a bunch of people and some of them lie to you and you figure out what’s true. So as long as the journalist treats the AI as yet another unreliable source, they’re actually better equipped to work with AI than most other professions are.

The pelican benchmark

Of course we talked about pelicans riding bicycles:

56:10—There appears to be a very strong correlation between how good their drawing of a pelican riding a bicycle is and how good they are at everything else. And nobody can explain to me why that is. [...]

People kept on asking me, what if labs cheat on the benchmark? And my answer has always been, really, all I want from life is a really good picture of a pelican riding a bicycle. And if I can trick every AI lab in the world into cheating on benchmarks to get it, then that just achieves my goal.

59:56—I think something people often miss is that this space is inherently funny. The fact that we have these incredibly expensive, power hungry, supposedly the most advanced computers of all time. And if you ask them to draw a pelican on a bicycle, it looks like a five-year-old drew it. That's really funny to me.

And finally, some good news about parrots

Lenny asked if I had anything else I wanted to leave listeners with to wrap up the show, so I went with the best piece of news in the world right now.

1:38:10—There is a rare parrot in New Zealand called the Kākāpō. There are only 250 of these parrots left in the world. They are flightless nocturnal parrots—beautiful green dumpy looking things. And the good news is they're having a fantastic breeding season in 2026,

They only breed when the Rimu trees in New Zealand have a mass fruiting season, and the Rimu trees haven't done that since 2022—so there has not been a single baby kākāpō born in four years.

This year, the Rimu trees are in fruit. The kākāpō are breeding. There have been dozens of new chicks born. It's a really, really good time. It's great news for rare New Zealand parrots and you should look them up because they're delightful.

Everyone should [watch the live stream of Rakiura on her nest with two chicks!](#)

YouTube chapters

Here's the full list of chapters Lenny's team defined for the YouTube video:

- 00:00: Introduction to Simon Willison
- 02:40: The November 2025 inflection point
- 08:01: What's possible now with AI coding
- 10:42: Vibe coding vs. agentic engineering
- 13:57: The dark-factory pattern
- 20:41: Where bottlenecks have shifted

- [23:36](#): Where human brains will continue to be valuable
- [25:32](#): Defending of software engineers
- [29:12](#): Why experienced engineers get better results
- [30:48](#): Advice for avoiding the permanent underclass
- [33:52](#): Leaning into AI to amplify your skills
- [35:12](#): Why Simon says he's working harder than ever
- [37:23](#): The market for pre-2022 human-written code
- [40:01](#): Prediction: 50% of engineers writing 95% AI code by the end of 2026
- [44:34](#): The impact of cheap code
- [48:27](#): Simon's AI stack
- [54:08](#): Using AI for research
- [55:12](#): The pelican-riding-a-bicycle benchmark
- [59:01](#): The inherent ridiculousness of AI
- [1:00:52](#): Hoarding things you know how to do
- [1:08:21](#): Red/green TDD pattern for better AI code
- [1:14:43](#): Starting projects with good templates
- [1:16:31](#): The lethal trifecta and prompt injection
- [1:21:53](#): Why 97% effectiveness is a failing grade
- [1:25:19](#): The normalization of deviance
- [1:28:32](#): OpenClaw: the security nightmare everyone is looking past
- [1:34:22](#): What's next for Simon
- [1:36:47](#): Zero-deliverable consulting

- [1:38:05](#): Good news about Kakapo parrots

A PM's Guide to AI Agent Architecture: Why Capability Doesn't Equal Adoption

UMANG · 05 SEP 2025 · [SOURCE](#)

Last week, I was talking to a PM who'd in the recent months shipped their AI agent. The metrics looked great: 89% accuracy, sub-second respond times, positive user feedback in surveys. But users were abandoning the agent after their first real problem, like a user with both a billing dispute and a locked account.

"Our agent could handle routine requests perfectly, but when faced with complex issues, users would try once, get frustrated, and immediately ask for a human."

This pattern is observed across every product team that focuses on making their agents "smarter" when the real challenge is making architectural decisions that shape how users experience and begin to trust the agent.

In this post, I'm going to walk you through the different layers of AI agent architecture. How your product decisions determine whether users trust your agent or abandon it. By the end of this, you'll understand why some agents feel "magical" while others feel "frustrating" and more importantly, how PMs should architect for the magical experience.

We'll use a concrete customer support agent example throughout, so you can see exactly how each architectural choice plays out in practice. We'll also see why the counterintuitive approach to trust (hint: it's not about being right more often) actually works better for user adoption.

Let's say you're building a customer support agent

You're the PM building an agent that helps users with account issues - password resets, billing questions, plan changes. Seems straightforward, right?



But when a user says *"I can't access my account and my subscription seems wrong"* what should happen?

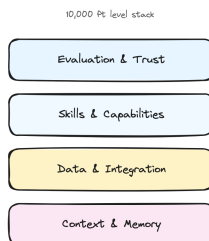
Scenario A: Your agent immediately starts checking systems. It looks up the account, identifies that the password was reset yesterday but the email never arrived, discovers a billing issue that downgraded the plan, explains exactly what happened, and offers to fix both issues with one click.

Scenario B: Your agent asks clarifying questions. "When did you last successfully log in? What error message do you see? Can you tell me more about the subscription issue?" After gathering info, it says "Let me escalate you to a human who can check your account and billing."

Same user request. Same underlying systems. Completely different products.

The Four Layers Where Your Product Decisions Live

Think of agent architecture like a stack where each layer represents a product decision you have to make.



Layer 1: Context & Memory (What does your agent remember?)

The Decision: How much should your agent remember, and for how long?

This isn't just technical storage - it's about creating the illusion of understanding. Your agent's memory determines whether it feels like talking to a robot or a knowledgeable colleague.

For our support agent: Do you store just the current conversation, or the customer's entire support history? Their product usage patterns? Previous complaints?

Types of memory to consider:

- **Session memory:** Current conversation ("You mentioned billing issues earlier...")
- **Customer memory:** Past interactions across sessions ("Last month you had a similar issue with...")
- **Behavioral memory:** Usage patterns ("I notice you typically use our mobile app...")
- **Contextual memory:** Current account state, active subscriptions, recent activity

The more your agent remembers, the more it can anticipate needs rather than just react to questions. Each layer of memory makes responses more intelligent but increases complexity and cost.

The Decision: Which systems should your agent connect to, and what level of access should it have?

The deeper your agent connects to user workflows and existing systems, the harder it becomes for users to switch. This layer determines whether you're a tool or a platform.

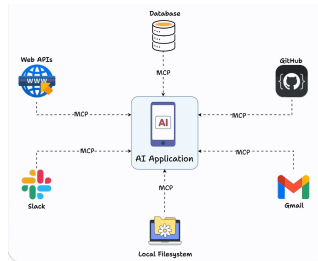
For our support agent: Should it integrate with just your Stripe's billing system, or also your Salesforce CRM, ZenDesk ticketing system, user database, and audit logs? Each integration makes the agent more useful but also creates more potential failure points - *think API rate limits, authentication challenges, and system downtime.*

Layer 3: Skills & Capabilities (What makes you different?)

The Decision: Which specific capabilities should your agent have, and how deep should they go?

Your skills layer is where you win or lose against competitors. It's not about having the most features - it's about having the right capabilities that create user dependency.

For our support agent: Should it only read account information, or should it also modify billing, reset passwords, and change plan settings? Each additional skill increases user value but also increases complexity and risk.



Implementation note: Tools like MCP (Model Context Protocol) are making it much easier to build and share skills across different agents, rather than rebuilding capabilities from scratch.

Layer 4: Evaluation & Trust (How do users know what to expect?)

The Decision: How do you measure success and communicate agent limitations to users?

This layer determines whether users develop confidence in your agent or abandon it after the first mistake. It's not just about being accurate - it's about being trustworthy.

For our support agent: Do you show confidence scores ("I'm 85% confident this will fix your issue")? Do you explain your reasoning ("I checked three systems and found...")? Do you always confirm before taking actions ("Should I reset your password now")? Each choice affects how users perceive reliability.

Trust strategies to consider:

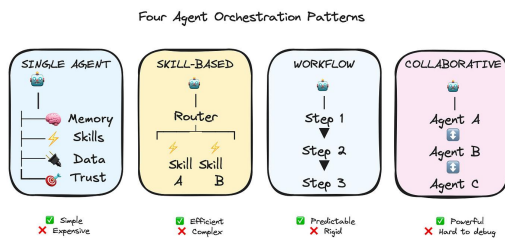
- **Confidence indicators:** "I'm confident about your account status, but let me double-check the billing details"
- **Reasoning transparency:** "I found two failed login attempts and an expired payment method"
- **Graceful boundaries:** "This looks like a complex billing issue - let me connect you with our billing specialist who has access to more tools"
- **Confirmation patterns:** When to ask permission vs. when to act and explain

The counterintuitive insight: users trust agents more when they admit uncertainty than when they confidently make mistakes.

So how do you actually architect an agent?

Okay, so you understand the layers. Now comes the practical question that every PM asks: "How do I actually implement this? How does the agent talk to the skills? How do skills access data? How does evaluation happen while users are waiting?"

Your orchestration choice determines everything about your development experience, your debugging process, and your ability to iterate quickly.



Lets walk through the main approaches, and I'll be honest about when each one works and when it becomes a nightmare.

1. Single-Agent Architecture (Start Here)

Everything happens in one agent's context.

For our support agent: *When the user says "I can't access my account," one agent handles it all - checking account status, identifying billing issues, explaining what happened, offering solutions.*

Why this works: Simple to build, easy to debug, predictable costs. You know exactly what your agent can and can't do.

Why it doesn't: Can get expensive with complex requests since you're loading full context every time. Hard to optimize specific parts.

Most teams start here, and honestly, many never need to move beyond it. If you're debating between this and something more complex, start here.

You have a router that figures out what the user needs, then hands off to specialized skills.

For our support agent: *Router realizes this is an account access issue and routes to the `LoginSkill`. If the LoginSkill discovers it's actually a billing problem, it hands off to `BillingSkill`.*

Real example flow:

1. User: "I can't log in"
2. Router → LoginSkill
3. LoginSkill checks: Account exists ✓, Password correct ✗, Billing status... wait, subscription expired
4. LoginSkill → BillingSkill: "Handle expired subscription for user123"
5. BillingSkill handles renewal process

Why this works: More efficient - you can use cheaper models for simple skills, expensive models for complex reasoning. Each skill can be optimized independently.

Why it doesn't: Coordination between skills gets tricky fast. Who decides when to hand off? How do skills share context?

Here's where MCP really helps - it standardizes how skills expose their capabilities, so your router knows what each skill can do without manually maintaining that mapping.

You predefine step-by-step processes for common scenarios. Think LangGraph, CrewAI, AutoGen, N8N, etc.

For our support agent: *"Account access problem" triggers a workflow:*

1. Check account status
2. If locked, check failed login attempts
3. If too many failures, check billing status
4. If billing issue, route to payment recovery
5. If not billing, route to password reset

Why this works: Everything is predictable and auditable. Perfect for compliance-heavy industries. Easy to optimize each step.

Why it doesn't: When users have weird edge cases that don't fit your predefined workflows, you're stuck. Feels rigid to users.

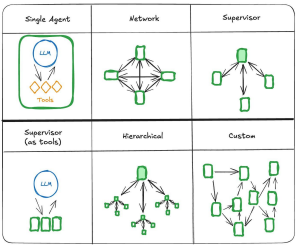
Multiple specialized agents work together using A2A (agent-to-agent) protocols.

The vision: Your agent discovers that another company's agent can help with issues, automatically establishes a secure connection, and collaborates to solve the customer's problem. *Think a booking.com agent interacting with an American Airlines agent!*

For our support agent: ``AuthenticationAgent`` handles login issues, ``BillingAgent`` handles payment problems, ``CommunicationAgent`` manages user interaction. They coordinate through standardized protocols to solve complex problems.

Reality check: This sounds amazing but introduces complexity around security, billing, trust, and reliability that most companies aren't ready for. We're still figuring out the standards.

This can produce amazing results for sophisticated scenarios, but debugging multi-agent conversations is genuinely hard. When something goes wrong, figuring out which agent made the mistake and why is like detective work.



But here's what's interesting - even with the perfect architecture, your agent can still fail if users don't trust it. That brings us to the most counterintuitive lesson about building agents.

The trust thing that everyone gets wrong

Here's something counterintuitive: Users don't trust agents that are right all the time. They trust agents that are honest about when they might be wrong.

Think about it from the user's perspective. Your support agent confidently says "I've reset your password and updated your billing address." User thinks "great!" Then they try to log in and... it doesn't work. Now they don't just have a technical problem - they have a trust problem.

Compare that to an agent that says "I think I found the issue with your account. I'm 80% confident this will fix it. I'm going to reset your password and update your billing address. If this doesn't work, I'll immediately escalate to a human who can dive deeper."

Same technical capability. Completely different user experience.

Building trusted agents requires focus on three things:

1. **Confidence calibration:** When your agent says it's 60% confident, it should be right about 60% of the time. Not 90%, not 30%. Actual 60%.
2. **Reasoning transparency:** Users want to see the agent's work. "I checked your account status (active), billing history (payment failed yesterday), and login attempts (locked after 3 failed attempts). The issue seems to be..."
3. **Graceful escalation:** When your agent hits its limits, how does it hand off? A smooth transition to a human with full context is much better than "I can't help with that."

A lot of times we obsess over making agents more accurate, when what users actually want was more transparency about the agent's limitations.

What's Coming Next

In Part 2, I'll dive deeper into the autonomy decisions that keep most PMs up at night. How much independence should you give your agent? When should it ask for permission vs forgiveness? How do you balance automation with user control?

We'll also walk through the governance concerns that actually matter in practice - not just theoretical security issues, but the real implementation challenges that can make or break your launch timeline.